NEUTRAL TASK BROKER

OpenVow 设计文档
合订本 v1.1

人定契约，终端干活，平台只做派发、校验、归集、合并。
没有隐含动作，只有任务；判卷的人不是考生。

共 18 册 · 2026-10
适用 OpenVow 1.x

OpenVow 设计文档集

> 这是一套分册文档，不是一个文件。每一册只讲一件事，册与册之间用编号互相引用。 所有章节号（§1–§22）是**全局唯一的锚点**，任何一册里出现的 `§9.2` 都指向本集内的同一处，不会随文件位置变化。

项	内容
版本	v1.1（2026-10-21）
状态	定稿，可据此施工
适用	OpenVow 1.x
读者	实现者、集成方（Harness 适配）、项目负责人
替代	合并并取代《项目想法 1.0 / 2.0 / 2.1 / 3.0》
合订本	`OpenVow-设计文档-合订本.pdf`（由本目录自动生成，勿手改）

---

1. 文档地图

#	文件	覆盖	一句话
—	README.md	—	索引、阅读路径、维护规则
02	02-概述与范围.md	§2–§3	要解决什么、不做什么、什么前提成立
03	03-术语.md	§4	图、契约、冻结、红测试等 15 个词的定义
04	04-总体架构.md	§5	三层架构、四类参与者、一次完整循环、部署形态
05	05-领域模型.md	§6	实体关系、完整 Postgres DDL、两套状态机
06	06-任务类型.md	§7	六种 kind 各自的写集、验收与依赖
07	07-契约体系.md	§8	任务契约、接口契约、验收冻结、图契约
08	08-校验体系.md	§9	V1–V8 图校验、L0–L3 产物校验、反作弊矩阵
09	09-调度与配额.md	§10	认领 SQL、租约心跳、信任分、热文件、fair-share
10	10-协议与接入.md	§11	端点总表、报文、错误码、MCP 适配器
11	11-安全与审计.md	§12	三种凭证、集成容器威胁模型、审计字段
12	12-成本与风险.md	§13–§14	成本量级与风险登记册
13	13-路线图与实现基线.md	§15–§16	M0–M4 验收标准、M0 技术基线

#	文件	覆盖	一句话
14	14-决策与未决.md	§17–§18	13 条 ADR 与 6 个未决问题
15	15-设计问题.md	§19	34 个设计问题、8 个没有答案的、3 个前提
16	16-验证实验.md	§20	六个生命体征、9 个可证伪实验、回写表
17	17-附录与结语.md	§21–§22	错误码全集、配置项、性能基线、结语
—	M0-施工清单.md	§16 细化	表结构、流水线、CLI、十天排期
—	架构图 3.0 · PDF	可视化	一页看完全局

> 编号按内容分组，不要求连续：第二章的文件叫 `02-…`，第十三章覆盖 §15–§16，是刻意的。

---

2. 按目的读

你想…	读这几册
判断这件事值不值得做	02 → 12 → 15
理解系统怎么转	04 → 05 → 06
动手写代码	05 → 07 → 08 → 09 → 13
接入一个 Harness	10 §10.4 → 06
做安全评审	11 → 08 §8.3
想知道哪里还没想清楚	15
想用半天证伪这套设计	16 实验 A + B

---

3. 按问题查

问题	答案在哪
为什么平台不能自己调模型？	02 §2.4、14 ADR-02
两个任务改同一个文件怎么办？	09 §10.1、08 §9.1
怎么防止 Agent 把测试改软？	06 §7.3、08 §9.3
接口为什么必须先冻结？	06 §7.2
空测试怎么被挡住？	06 §7.3（红测试判定四条）

问题	答案在哪
合并失败了谁修?	06 §7.6（repair）
终端失联了任务怎么回收?	09 §10.2
平台怎么防陌生代码?	11 §12.2
多个人共享算力怎么分?	09 §10.5（fair-share，不是奖励）
怎么对接 Claude Code?	10 §10.4（MCP 适配器）
第一个版本做多大?	13 §15（M0）

---

4. 文档之间的关系

```
README（索引）
   │
   ├── 为什么做 ──────── 02 概述与范围
   │                        ├── 03 术语
   │                        └── 12 成本与风险
   │
   ├── 怎么做 ────────── 04 总体架构
   │                        ├── 05 领域模型     ← 数据与状态
   │                        ├── 06 任务类型     ← 谁干什么
   │                        ├── 07 契约体系     ← 干什么算合格
   │                        ├── 08 校验体系     ← 怎么判合格
   │                        ├── 09 调度与配额   ← 谁先干
   │                        ├── 10 协议与接入   ← 怎么连
   │                        └── 11 安全与审计
   │
   ├── 什么时候做 ──── 13 路线图与实现基线
   │                        └── M0-施工清单
   │
   ├── 为什么这样定 ── 14 决策与未决
   │
   └── 哪里还没底 ──── 15 设计问题
                            └── 16 验证实验
                                    └── 17 附录与结语
```

---

5. 维护规则

1. **章节号是锚点，不随文件移动。** 一册可以拆、可以合，但 `§9.2` 永远指校验体系的第二小节。新增内容应插在所属章节内部（§9.2.1），不要重排全局编号。
2. **单一真源。** 同一事实只写在一册里，其他地方用链接引用。发现两处都在描述同一件事，就是该合并的信号。
3. **合订本由工具生成。**

   ```
   python tools/build-pdf.py
   ```

   它会按文件名顺序拼接本目录下所有 `*.md`（README 在最前），生成 `docs/OpenVow-设计文档-合订本.pdf`。**不要手改 PDF。**
4. **改完文档，检查三件事**：交叉引用的编号是否还有效、被替换的常量是否同步（§20.7 那张表）、合订本是否重出。
5. **假设必须标注。** 任何没有实验或数据支撑的结论，在文中要写成"假设"而不是"决定"。§19 的 34 个问题就是这条规则的产物。

---

6. 变更记录

版本	日期	变更
v1.1	2026-10-21	增补 §19 设计问题（34 问）与 §20 验证实验（9 个）；补 §10.5 配额；拆分为分册
v1.0	2026-10-21	合并 1.0–3.0，形成完整设计文档
3.0	2026-10-21	接口前置、红测试、集成即任务
2.1	2026-10-21	拆解层设计
2.0	2026-10-21	中立任务中台设计
1.0	2026-10-21	最初的想法

概述与范围

> OpenVow 设计文档集 ｜ v1.1 ｜ 覆盖 §2、§3
> 下一章：术语 ｜ 索引

---

2. 概述

2.1 一句话

OpenVow 是一个私有工程的任务中台：人定契约，终端干活，平台只做派发、校验、归集、合并。

2.2 要解决的三个问题

#	问题	现状
P1	大需求只能串行做	一个 Agent 会话从头做到尾，墙钟时间等于全部工作量，且中途崩溃就从零重来
P2	并行产出合不起来	多个 Agent 各自发明接口、各自写测试、各自用 stub 自测通过，合并时接口对不上、测试被改软
P3	多账号调用被集中管控	一旦做成"平台调模型"，所有 Token、限流、封号风险都压回中心，且成本不可分摊

多数"多 Agent 编排"产品把 P1 当成唯一问题，用并行解决它，结果把 P2 放大成灾难：**产出的代码多了 5 倍，能合并的没有变多。**

2.3 解题思路

```
把「能不能并行」变成「契约能不能先冻结」
把「能不能合并」变成「判卷的人是不是考生」
把「谁在调模型」变成「终端自己调，平台永不见 Key」
```

对应三个机制：

1. **前置契约**：动手之前，接口、测试、验收命令全部冻结，产物不需要二次加工。
2. **独立判卷**：测试由独立任务产出、平台冻结执行，实现方无法给自己打分。
3. **合并即任务**：集成不是流水线上一个勾选步骤，而是一次真实的构建与测试执行，失败会生成修复任务。

2.4 五条设计原则

#	原则	推论
1	**平台不持有模型凭证**	任何需要模型的动作，都必须由终端完成
2	**平台不主动调用**	只有终端拉取，没有平台推送
3	**平台不改写产物**	只做校验、归档、按序合并，不做修补
4	**没有隐含动作，只有任务**	拆解、写接口、写测试、写实现、合并、修复，都是任务
5	**判卷的人不是考生**	验收产物与实现产物必须来自不同任务，且前者被冻结

3. 范围与前提

3.1 做什么

- 把一个较大的软件需求，切成**文件不相交、接口已冻结、验收可执行**的子任务；
- 分发给多个自带模型账号的终端，并行执行；
- 对每个产物做结构性校验与行为验收；
- 把合格产物按 DAG 顺序合并进一条可编译的主干；
- 全过程可追溯：谁、何时、基于什么基线、用什么模型、通过了什么。

3.2 不做什么（scope guard）

不做	原因
单文件小改动	建契约的成本大于改代码本身
强耦合的存量改造	热文件会把并行度压到 1（§10.4）
架构设计决策	平台明确不做设计判断，`decisions[]` 必须由人给
替代人工 code review	平台只保证「结构合法 + 验收通过」，不保证「设计正确」
需求管理 / 版本规划 / 发布	它只管一次需求到合并
通用 CI	只跑与任务相关的构建与测试，不做流水线编排
公共 Agent 集市	私有工程场景，不做匿名撮合、信誉交易
激励、代币、结算	私有场景的痛点是分配不是激励；可交易的积分必然被刷（§10.5）

3.3 三个成立前提

1. **有 ≥3 个可独立干活的执行体**（人 / 账号 / 机器）。只有 1 个，分布式的全部机制都是净开销。
2. **工作以新增文件、新增模块为主。**改造存量时，注册表、路由、清单这些热文件会把 DAG 串行化。
3. **接受「平台只保证结构与验收，不保证内容质量」**，并有评审带宽兜底。

3.4 回本门槛

指标	低于此值不建议上	说明
独立执行体	3	2 个的收益抵不上协调成本
每周子任务数	15	更低时人工串行更快
单任务规模	100–400 行	太小则契约成本占比过高，太大则失败代价过高

3.5 与现有方案的关系

- **不是**任务队列（Airflow / Celery）：它们调度程序，不处理"谁的接口对不上"。
- **不是** Agent 编排框架（LangGraph / CrewAI）：它们由平台调用模型，风控与成本集中。
- **不是** A2A / Agent 集市：那是公共网络的任务撮合，没有代码级契约。
- **是**「私有工程 + 代码契约 + 终端主动拉取」的三元组合。单点都不新，组合是新的。

4. 术语

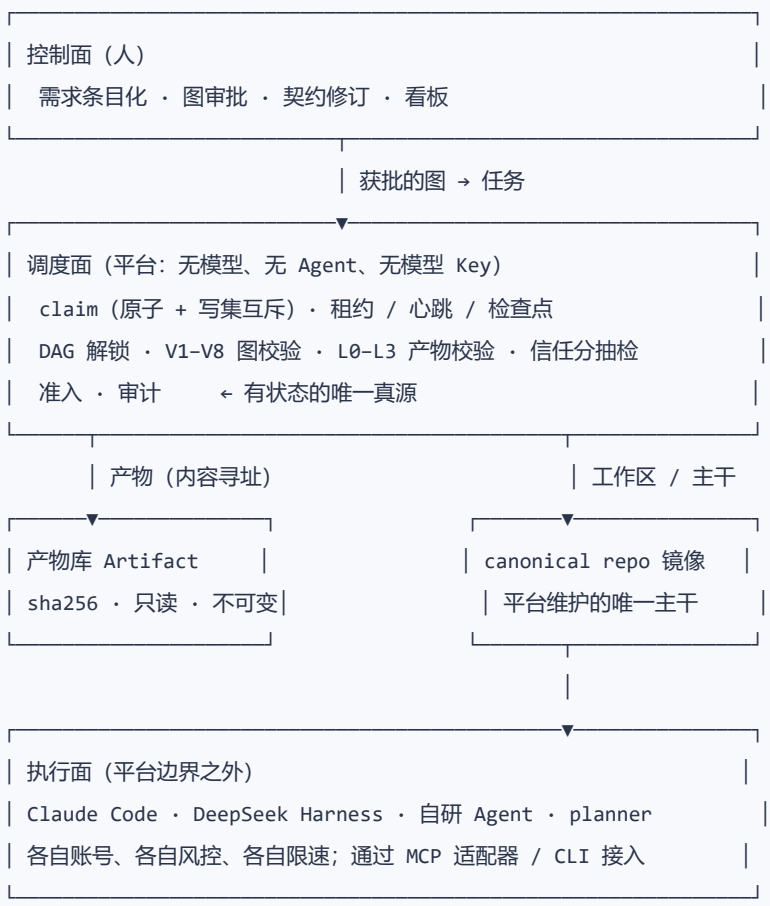
> OpenVow 设计文档集 | v1.1 | 覆盖 §4
> 上一章：概述与范围 | 下一章：总体架构 | 索引

术语	含义
需求条目	需求文档中带稳定 id 的最小可验证单元，如 `R-14/2.1`，被 `covers` 引用
图（graph）	一张 `task_graph.json`，描述一次需求的全部任务、依赖与覆盖关系
任务（task）	图中的一个节点，或平台生成的集成 / 修复任务
契约（contract）	任务的写集、接口、验收、覆盖四件套
写集（writes）	任务允许写入的路径集合（glob），任务间必须两两不相交
冻结（frozen）	接口与测试在 accepted 后不可再改，平台以其哈希为准
红测试（red tests）	在骨架上必须失败的测试，用来证明它不是空测试
一次通过（first-pass）	首次提交即 accepted，不消耗 attempts
merge group	一起合并的一组任务，默认整图，可拆
集成任务（integration）	把 group 产物按序应用并全量验证的任务
失败包（failure bundle）	集成失败时生成的上下文，作为 repair 任务的输入
canonical repo	平台维护的唯一主干，只有集成流程可写
信任分（trust）	终端近 20 次提交的一次通过率，决定平台复跑比例
外部终端	平台边界之外的执行体（Claude Code / DSH / 自研 Agent）

5. 总体架构

> OpenVow 设计文档集 | v1.1 | 覆盖 §5
> 上一章：术语 | 下一章：领域模型 | 索引

5.1 分层



5.2 参与者

参与者	职责	凭证
人	写需求条目、批准图、修订契约、处理 `needs_human`	human token
planner 终端	把需求拆成图	terminal token（scope=planner）
执行终端	领 interface / test / implement / integration / repair	terminal token（scope=worker）
平台	派发、校验、归集、合并、审计	无模型凭证

5.3 一次完整循环

1. 人把需求写成条目化文档（每条有稳定 id）。
2. 人发一个 `decompose` 任务。
3. planner 终端 claim，读需求 + 仓库快照 + `decisions[]`，产出 `task_graph.json` 草案。
4. 平台跑 V1–V8，把 `violations[]` 返回给 planner；不通过就改。
5. 人审 diff，局部改或驳回；通过后图 `active`。
6. 平台把图展开成任务，按拓扑排布；`integration` 由平台生成、置于末尾。
7. 各终端独立 claim。**同时被认领的任务写集两两不相交。**
8. 终端提交产物与证据；平台跑 L0–L3，通过则 `accepted`，解锁下游。
9. 一个 merge group 全部 `accepted` 后，`integration` 解锁。
10. 某终端 claim 集成，按 DAG 顺序 apply 到临时分支，容器内全量构建 + 测试。
11. 通过 → fast-forward 到主干，`merged`；失败 → 生成 `repair` 任务，回到第 7 步。
12. 全部 `merged` 后需求闭环；看板记录墙钟、成本、返工。

5.4 部署形态

形态	组成	适用
T0 单机	单二进制 + SQLite + 本地目录	M0 / M1，单终端
T1 服务化	api + Postgres + 对象存储（MinIO/S3）+ 反向代理	M2 起，多终端
T2 隔离	T1 + 独立 runner 主机 + 容器运行时	M3 起，跑不可信构建

平台自身永远不挂模型 SDK，镜像里不应出现任何模型供应商的依赖——这是可以被审计的硬约束。

5.5 代码同步与 canonical repo

终端必须能看到别人已 `accepted` 的代码，否则每个任务都在对过期快照工作。

项	设计
镜像	平台维护 `projects(mirror_url, canonical_ref, head_commit)`，从 origin 单向拉取
认领	`claim` 返回 `repo_url` 与 `base_commit`；终端自行 fetch 后 checkout
提交	终端**不 push**，只交文件哈希；二进制走 artifact 上传
合并	只有集成任务能把产物写成 commit；平台只决定顺序，不改字节
回传	可选推回 origin，默认只留在平台主干

6. 领域模型

> OpenVow 设计文档集 | v1.1 | 覆盖 §6
> 上一章：总体架构 | 下一章：任务类型 | 索引

6.1 实体关系

```
project 1—n graph 1—n task n—n task (task_deps)
                  task 1—n submission
                  task 1—n task_files n—1 artifact
                  task 1—n checkpoint
                  task 1—n event
project 1—n terminal
integration 1—n integration_record
```

6.2 数据模型（PostgreSQL 规范版）

```
-- 项目与仓库镜像
CREATE TABLE projects (
  id            text PRIMARY KEY,
  name          text NOT NULL,
  repo_url      text NOT NULL,
  canonical_ref text NOT NULL DEFAULT 'main',
  head_commit   text,
  created_at    timestamptz NOT NULL DEFAULT now()
);

-- 终端（外部执行体）
CREATE TABLE terminals (
  id           text PRIMARY KEY,
  project_id   text REFERENCES projects(id),
  name         text NOT NULL,
  scope        text NOT NULL CHECK (scope IN ('planner','worker','admin')),
  token_hash   bytea NOT NULL,
  labels       text[] NOT NULL DEFAULT '{}',
  trust        numeric(4,3) NOT NULL DEFAULT 0,
  accepted_n   int NOT NULL DEFAULT 0,
  total_n      int NOT NULL DEFAULT 0,
  banned_at    timestamptz,
  last_seen_at timestamptz
);

-- 图（每次拆解一个版本，永不物理删除）
CREATE TABLE graphs (
  id              text PRIMARY KEY,
  project_id      text NOT NULL REFERENCES projects(id),
  version         int  NOT NULL,
  state           text NOT NULL CHECK (state IN ('draft','validated','approved','active','superseded','rejected'
  requirement_ref text NOT NULL,
  base_commit     text NOT NULL,
  body            jsonb NOT NULL,          -- task_graph.json 原文
  sha256          text NOT NULL,
  supersedes      text REFERENCES graphs(id),
  approved_by     text,
  approved_at     timestamptz,
  created_at      timestamptz NOT NULL DEFAULT now(),
  UNIQUE (project_id, version)
);

-- 任务
CREATE TABLE tasks (
```

```
  id           text PRIMARY KEY,
  graph_id     text REFERENCES graphs(id),
  node_id      text,                        -- 图内节点 id；平台生成的为空
  kind         text NOT NULL CHECK (kind IN
                 ('decompose','interface','test','implement','integration','repair')),
  title        text NOT NULL,
  writes       text[] NOT NULL,             -- 必须两两不相交
  reads        text[] NOT NULL DEFAULT '{}',
  provides     text[] NOT NULL DEFAULT '{}',
  consumes     text[] NOT NULL DEFAULT '{}',
  covers       text[] NOT NULL DEFAULT '{}',
  frozen       jsonb  NOT NULL DEFAULT '[]',-- [{"path","sha256"}]
  acceptance   jsonb  NOT NULL,
  requires     text[] NOT NULL DEFAULT '{}',
  state        text NOT NULL CHECK (state IN
                 ('queued','claimed','validating','accepted','rejected','merged','stale','needs_human')),
  attempts     int NOT NULL DEFAULT 0,
  max_attempts int NOT NULL DEFAULT 2,
  priority     int NOT NULL DEFAULT 100,
  lease_id     text,
  lease_until  timestamptz,
  terminal_id  text REFERENCES terminals(id),
  base_commit  text,
  merge_group  text,
  created_at   timestamptz NOT NULL DEFAULT now(),
  updated_at   timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX tasks_claim_idx  ON tasks (state, priority DESC, created_at);
CREATE INDEX tasks_writes_idx ON tasks USING gin (writes);   -- 支撑 && 交叠判断

CREATE TABLE task_deps (
  parent_id text NOT NULL REFERENCES tasks(id),
  child_id  text NOT NULL REFERENCES tasks(id),
  PRIMARY KEY (parent_id, child_id)
);

-- 中间产物（断点续跑）
CREATE TABLE checkpoints (
  id         bigserial PRIMARY KEY,
  task_id    text NOT NULL REFERENCES tasks(id),
  attempt    int  NOT NULL,
  seq        int  NOT NULL,
  note       text,
  files      jsonb NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

```
-- 内容寻址产物库
CREATE TABLE artifacts (
  sha256     text PRIMARY KEY,
  size       bigint NOT NULL,
  media_type text,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE task_files (
  task_id text NOT NULL,
  attempt int  NOT NULL,
  path    text NOT NULL,
  sha256  text NOT NULL REFERENCES artifacts(sha256),
  PRIMARY KEY (task_id, attempt, path)
);

-- 提交记录（幂等键）
CREATE TABLE submissions (
  id         bigserial PRIMARY KEY,
  task_id    text NOT NULL,
  attempt    int  NOT NULL,
  lease_id   text NOT NULL,
  accepted   boolean NOT NULL,
  code       text,
  detail     jsonb,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (task_id, attempt)
);

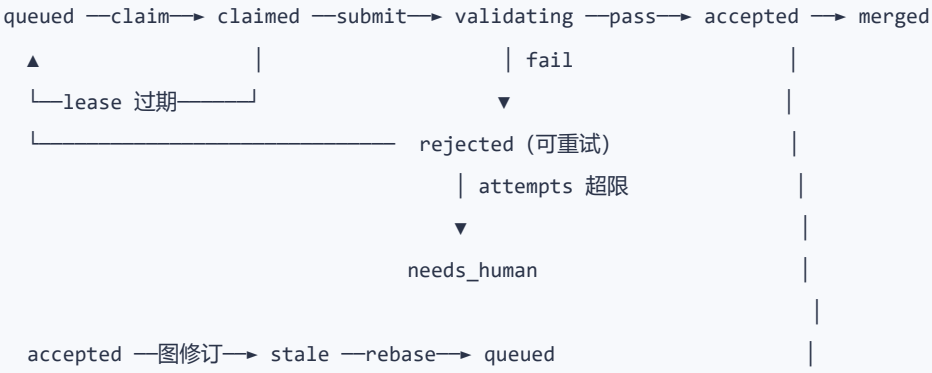
-- 集成记录
CREATE TABLE integrations (
  id               bigserial PRIMARY KEY,
  merge_group      text NOT NULL,
  candidate_commit text,
  result           text NOT NULL CHECK (result IN ('merged','failed','timeout')),
  bundle           jsonb,
  created_at       timestamptz NOT NULL DEFAULT now()
);

-- 审计（只追加）
CREATE TABLE events (
  id      bigserial PRIMARY KEY,
  task_id text,
  kind    text NOT NULL,
  payload jsonb,
```

```
  at      timestamptz NOT NULL DEFAULT now()
);
```

SQLite 差异（M0）：`text[]` → JSON 文本；`jsonb` → `TEXT`；`timestamptz` → `TEXT`（RFC3339 UTC）；`bigserial` → `INTEGER PRIMARY KEY AUTOINCREMENT`；无 `&&` 运算与 GIN 索引（M0 不并行，不需要）。

6.3 任务状态机



规则：

- `claimed` 必须有未过期的 `lease_id`；
- `accepted` 是唯一的解锁条件（不是 `done`、不是终端自报 success）；
- `stale` 的产物禁止进入集成，但不删除；
- `needs_human` 是终态，只能由人重置。

6.4 图状态机



规则：

- 只有 `active` 图下的任务可派发；
- 任何修订产生新 `version`，旧版 `superseded`，**永不物理删除**；
- `approve` 必须由 human token 调用，planner 无此权限。

7. 任务类型

> OpenVow 设计文档集 | v1.1 | 覆盖 §7
> 上一章：领域模型 | 下一章：契约体系 | 索引

7.1 六种类型

kind	目的	写集	验收	依赖	产物
`decompose`	需求 → 图	∅	`graph_valid`	无	`task_graph.json`
`interface`	冻结接口骨架	接口文件（独占）	`skeleton_build`	decompose 批准	骨架 + 签名清单
`test`	产出验收测试	测试 + fixtures	`red_tests`	interface	测试源码 + 红证据
`implement`	实现功能	实现文件	`green_tests`	test	实现源码
`integration`	合并 + 全量验证	全仓（受控）	`full`	组内全 accepted	合并 commit + 报告
`repair`	修集成失败	冲突相关文件	`full`	integration failed	修复 diff

7.2 `interface`：接口前置，独占，只读

问题：两个并行任务各自发明一套 API，各自用 stub 自测通过，合并时对不上。

做法：

- 每个模块至少一个 `interface` 任务，产出**可编译的骨架**：
  - Rust `pub fn parse(s: &str) -> Expr { todo!() }`
  - TypeScript `export function parse(s: string): Expr { throw new Error("not implemented") }`
  - Python `def parse(s: str) -> Expr: raise NotImplementedError`
  - 以及 `.proto` / `.d.ts` / `trait` / `interface` 定义。
- 骨架必须编译通过，否则下游测试写不出来。
- 该任务 `writes` 即接口文件集合；`accepted` 后，这些路径**全图只读**：
  - 后续任务 `writes` 命中 → V2 拒绝；
  - 提交命中 → L0 拒绝；
  - 要改 → 走图修订（新 interface 版本），下游标 `stale`。
- 产出 `openvow.interface.json` 签名清单，由语言提取器生成，不让模型手写。

验收 `skeleton_build`：只要求构建通过，不跑逻辑测试（骨架里全是 `todo!()`，跑了必然 panic）。

为什么必须先做：它把"接口长什么样"从隐式约定变成显式产物。没有它，`decisions[]` 只能覆盖人想到的部分，剩余分歧全部堆到集成阶段爆炸。

7.3 `test`：独立任务，红测试验收

问题：实现方自己写测试，只要把断言改软就能通关。

做法：

- 测试由独立任务产出，`depends_on` 指向 `interface`，`writes` 只含测试与 fixtures；
- 实现任务 `depends_on` 指向测试，且**测试文件不在实现写集里**；
- 平台复跑实现任务时，**用冻结的测试覆盖终端交上来的测试。**

验收 `red_tests`：测试在骨架上**必须失败。**

```
{
  "kind": "red_tests",
  "build": "cargo test --locked --no-run",
  "run": "cargo test --locked",
  "expect": "fail",
  "min_failing_ratio": 0.8,
  "timeout_s": 900
}
```

判定（全部满足）：

1. `build` 退出码 0 —— 测试能编译；
2. `run` 退出码非 0 —— 确有失败；
3. 失败用例 / 总数 ≥ 0.8 —— 不是"一个红灯、其余全绿"的假红；
4. 失败原因是断言或 `not implemented` panic，不是编译错误。

为什么能防住空测试：`assert!(true)` 在骨架上是通过的，于是第 2 条不成立、整单被拒。想让测试被接受，唯一的办法是写出针对未实现行为会失败的断言。

局限：它防"空洞测试"，防不住"测试写错了需求"。后者靠 V4 覆盖度 + 人工抽检。

7.4 `implement`

- `writes` = 实现文件；接口与测试文件不在其中；
- 验收 `green_tests`：冻结测试全绿 + lint 零 error；
- 提交若包含测试文件 → L1 拒绝 `frozen_file_modified`。

7.5 `integration`：合并是一次执行

- **谁创建**：平台自动物化，不由 planner 写进图。规则：merge group 成员全部 `accepted` 后生成。
- **写集**：组内所有节点写集并集 + 热文件。此时其它任务都已 accepted，不存在冲突。
- **并发**：全局串行（同一 canonical 分支同一时刻只允许一个），由 SQL 约束保证。
- **执行**：claim 的终端拿到 bundle（组内全部产物 + 顺序），在临时分支按 DAG 顺序 apply，得 `candidate_commit`，容器内全量构建 + 测试。
- **通过**：fast-forward 到主干，组内任务标 `merged`。
- **失败**：生成失败包，创建 `repair` 任务。

验收 `full`：全量构建 + 全量测试，超时 3600s。

为什么由终端跑：合并可能需要解决语义冲突（两个任务都对，组合起来类型不匹配），不是机械操作。平台只负责"按序应用 + 跑测试"。

7.6 `repair`

- 由集成失败触发，输入是失败包：`candidate_commit`、失败测试名、日志片段、相关 diff。
- `writes`：默认 = 失败测试覆盖到的文件；无法定位时放开到整个 group。
- 终端选择：优先派给**没有参与产出这些文件**的终端（避免作者盲区）。
- `accepted` 后，integration 以新 attempt 重跑；最多 3 次，超过 → `needs_human`。

7.7 `decompose`

- 输入：需求条目、`base_commit`、`decisions[]`、历史图的正/负样本；
- 输出：`task_graph.json`（只输出这个）；
- 验收 `graph_valid`：V1–V8 全过；
- 图不占任务池，未批准前执行终端不可见。

7.8 拓扑约束（V8）



- 任何 `implement` 的祖先链上必须有 `interface` 与 `test`；
- `test` 的依赖必须指向 `interface`；
- `integration` 不进图（平台生成），图里出现即报错；
- 同一模块必须有且只有一个 `interface`。

8. 契约体系

> OpenVow 设计文档集 | v1.1 | 覆盖 §8
> 上一章：任务类型 | 下一章：校验体系 | 索引

8.1 任务契约

```
{
  "id": "N-03",
  "kind": "implement",
  "title": "解析器主流程",
  "summary": "基于 N-01 的骨架实现 parse / parse_expr",
  "writes": ["src/parser/impl/**"],
  "reads": ["src/parser/mod.rs", "src/parser/ast.rs", "tests/parser/**"],
  "provides": [],
  "consumes": ["parser::parse", "parser::Expr"],
  "depends_on": ["N-01", "N-02"],
  "covers": ["R-14/2.1", "R-14/2.2"],
  "acceptance": {
    "kind": "green_tests",
    "build": "cargo build --locked",
    "run": "cargo test --locked parser",
    "expect": "pass",
    "timeout_s": 900
  },
  "est": { "files": 3, "diff_lines": 320 }
}
```

字段	规则
`writes`	相对仓库根 glob；两任务相交 → 拒；命中冻结接口路径 → 拒
`reads`	只影响终端取上下文，不参与冲突判定
`provides/consumes`	见 §8.2
`depends_on`	只表达"必须等它 accepted"
`covers`	必须引用需求条目 id；未覆盖会被 V4 点名
`acceptance.kind`	`graph_valid` / `skeleton_build` / `red_tests` / `green_tests` / `full`

8.2 接口契约

- `provides`：本任务承诺新增的符号（来自 `openvow.interface.json`）；
- `consumes`：本任务依赖的符号；
- **V7 闭合性**：每个 `consumes` 必须在同图某节点的 `provides` 里，或存在于 `base_commit` 的导出符号中。

诚实说明：V7 只做名字级匹配（符号存在 + 参数个数），不做完整类型检查。真正的类型一致性由 `green_tests` 的编译阶段兜底——这正是测试必须先写的原因。

8.3 验收冻结

冻结对象	何时冻结	怎么强制
接口文件	`interface` accepted	全图只读；提交命中即 L0 拒
测试文件	`test` accepted（记 sha256）	复跑用冻结版本覆盖；提交版本不同 → `frozen_file_modified`
验收命令	图批准	任务不能改自己的 `acceptance`

推论：**实现方永远无法给自己判卷。** 它能改的只有实现文件，判卷的是别人写的、被平台冻结的测试。

8.4 图契约

```
{
  "schema": "openvow/graph/v2",
  "requirement": {
    "id": "R-14",
    "title": "表达式解析器",
    "items": [
      { "id": "R-14/2.1", "text": "支持四则运算与括号" },
      { "id": "R-14/2.2", "text": "错误输入返回带位置的 ParseError" }
    ],
    "base_commit": "9c1f3ab"
  },
  "decisions": [
    { "id": "D-1", "text": "手写递归下降，不引入生成器", "affects": ["N-01"] }
  ],
  "nodes": ["…见 §8.1…"],
  "uncertain": ["需求未说明错误恢复策略"]
}
```

设计取舍：

- **不存** `edges`：依赖只在 `depends_on`，边由它推导，避免两处漂移；
- `decisions[]` **是人的领地**：planner 不得新增架构决策；

- `uncertain[]` **是逃生口**：让 planner 承认不确定，比逼它编一个完整图好。

9. 校验体系

> OpenVow 设计文档集 | v1.1 | 覆盖 §9
> 上一章：契约体系 | 下一章：调度与配额 | 索引

只剩两组：**V 组看图，L 组看产物**。合并是任务，不是校验阶段。

9.1 V 组（图级，纯函数，不调模型）

码	检查	典型 hint
`graph_schema_invalid`	不符合 JSON Schema	指出字段路径
`node_id_dup`	节点 id 重复	重命名
`dep_missing`	依赖指向不存在节点	列出合法 id
`dep_cycle`	有环	返回环序列
`write_overlap`	两节点写同一路径	合并或加串行
`write_outside_repo`	绝对路径 / 越出仓库	用相对根路径
`path_not_normalized`	`./`、`..`、`//`	规范化后重提
`acceptance_missing`	缺验收	每节点必须有
`acceptance_trivial`	`exit 0` / `true` / `--no-run` / 逻辑或恒真	空命令
`required_not_in_writes`	必填文件不在写集	补进 writes
`requirement_uncovered`	需求条目无覆盖	列出未覆盖条目
`covers_unknown_ref`	引用不存在的条目	列出合法引用
`too_many_files` / `node_too_big` / `too_many_nodes`	粒度超限	再拆或两级拆解
`base_commit_unknown`	基线不存在	用当前 HEAD 重拆
`writes_conflict_active`	与在途任务写集相交	排后或改边界
`interface_missing`	implement 祖先没有 interface	补节点
`interface_ambiguous`	同模块多个 interface	合并
`interface_write`	非 interface 节点写冻结接口	删掉这几行
`test_missing`	implement 祖先没有 test	补节点

码	检查	典型 hint
`test_write`	非 test 节点写测试路径	删掉
`consumes_unknown`	V7 符号未闭合	补 provides 或改签名
`kind_topology`	类型顺序错误	按 §7.8 重排
`decision_tampered`	`decisions[]` 被改	恢复

9.2 L 组（产物级）

层	检查	谁跑	失败
L0 结构	路径 ⊆ `writes`；无 `..`/绝对路径/符号链接；单文件 ≤ 2MB；总计 ≤ 50MB	平台	`rejected`
L1 完整	重算 sha256；`required_files` 齐全；冻结文件未被改动；编码统一	平台	`rejected`
L2 形状	`schema_output` 过 JSON Schema；linter 零 error	平台	`rejected`
L3 行为	按 `acceptance.kind` 执行	终端跑 + 平台按信任分复跑	`rejected` → 重试

9.3 反作弊矩阵

作弊手法	防御	层级
改软测试	测试独立任务 + 冻结 + 复跑覆盖	L1
空命令	静态黑名单 + `strength` + 复跑看输出	V3 / L3
空洞断言	**红测试**：空洞断言在骨架上通过 → 拒	L3
改接口蒙混	接口冻结 + 写集拒绝	V2 / L0
越界提交	路径 ⊆ writes	L0
旧基线覆盖	`base_commit` + `stale` + rebase	L1 / 集成
假日志	平台复跑（信任分）+ 日志结构校验	L3
声称成功	只认退出码与测试结果	L3

9.4 执行环境

场景	环境	网络
终端本地跑 L3	终端自己的机器	终端负责
平台复跑 L3	容器，10 分钟上限	断网 + 包代理白名单
集成全量	容器，60 分钟上限	同上

10. 调度

> OpenVow 设计文档集 | v1.1 | 覆盖 §10
> 上一章：校验体系 | 下一章：协议与接入 | 索引

10.1 认领 SQL（写集互斥必须在这里）

```
WITH candidate AS (
  SELECT t.id
  FROM tasks t
  WHERE t.state = 'queued'
    AND t.kind = ANY($1)                        -- 终端声明的能力
    AND NOT EXISTS (                            -- 前驱未 accepted
      SELECT 1 FROM task_deps d JOIN tasks p ON p.id = d.parent_id
      WHERE d.child_id = t.id AND p.state <> 'accepted'
    )
    AND NOT EXISTS (                            -- 写集互斥
      SELECT 1 FROM tasks o
      WHERE o.state = 'claimed' AND o.lease_until > now()
        AND o.writes && t.writes
    )
    AND NOT EXISTS (                            -- 集成全局串行
      SELECT 1 FROM tasks g
      WHERE g.kind IN ('integration','repair') AND g.state = 'claimed'
        AND t.kind IN ('integration','repair')
    )
  ORDER BY
    CASE t.kind WHEN 'integration' THEN 2 WHEN 'repair' THEN 3 ELSE 1 END,
    t.priority DESC, t.created_at
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
UPDATE tasks
SET state = 'claimed', terminal_id = $2, lease_id = $3,
    lease_until = now() + make_interval(secs => $4)
FROM candidate
WHERE tasks.id = candidate.id
RETURNING tasks.*;
```

两个工程要点：

1. `writes` 必须是 `text[]` + GIN 索引，`&&` 才能用；`jsonb` 走不通。
2. 写集互斥是**同一条语句里的谓词**。规范说 A、示例写 B，实现时一定按示例写。

10.2 租约、心跳、检查点

机制	规格
租约	默认 1800s；返回 `lease_expires_at`（服务端时钟）
心跳	每 300s；返回 `remaining_seconds`；`409` 时立即停手且不提交
过期	`claimed → queued`，`attempts += 1`（上限 2）→ `needs_human`
检查点	`POST /v1/checkpoint`，最多保留 5 个
续跑	重试时 `claim` 返回 `resume`，终端可从半成品继续
幂等	`submit` 按 `(task_id, attempt)` 幂等，重复提交返回首个结果

检查点解决"90 分钟白干"：租约只保证锁不死，不保证工作不丢。

10.3 信任分与抽检

```
trust = 近 20 次提交中「一次通过」的比例
```

条件	复跑比例
新终端（< 5 次）	100%
trust ≥ 0.95 且 n ≥ 20	5%
trust ≥ 0.90	20%
其它	50%

复跑未通过不直接判死（可能是环境差异），而是降 trust 并要求人工复核。没有这个旋钮，你要么全量复跑（变成 CI 农场），要么完全信任（等于没验收）。

10.4 热文件策略

注册表、路由表、`mod.rs`、`package.json`、DI 容器——任何新模块都要动它们。三种策略按优先级：

1. **消除**：约定式自动发现（glob 扫描、entrypoint 目录），新增模块不需要改中心文件。**唯一真正的解法。**
2. **隔离**：设计 `registrar` 任务，写集只含中心文件，放在图末尾串行执行。
3. **追加合并**：允许标记区间

```
// openvow:append:routes
// openvow:end
```

多个任务往各自区间追加，集成时按确定性顺序拼接。平台只拼接，不改内容。

三种都做不到，这个项目不适合 OpenVow。

10.5 贡献与配额（fair-share，不是奖励）

这一节解决的是分配公平，不是激励。

私有团队里会出现这种局面：A 挂了三台机器、三个账号，B 一台没挂但天天派活。半年后 A 会问"凭什么我出的算力给你跑需求"。这不是激励问题，是分配问题。

HPC 集群几十年前就遇到过同一件事，答案叫 **fair-share scheduling**（SLURM 的 priority multifactor）。它的核心不是发奖励，而是**记账 + 按历史用量调整优先级**。OpenVow 抄的是这个，不是积分商城。

三条边界

边界	含义
只记账，不结算	报表给人看，人对账；系统不碰钱。一旦碰钱就要有仲裁、发票、合规，成本会超过产品本身
只降权，不阻断	超配额只降低 claim 优先级，不硬拒绝。硬拒绝会让整张 DAG 卡死，比不公平更糟
只算 accepted	贡献口径用 `accepted` 任务的终端时长，不用提交次数，否则又诱导刷量

数据

```
CREATE TABLE owners (
  id     text PRIMARY KEY,
  name   text NOT NULL,
  period text NOT NULL DEFAULT 'week'       -- week / project
);

CREATE TABLE usage_ledger (
  owner_id         text NOT NULL REFERENCES owners(id),
  period_key       text NOT NULL,           -- 如 2026-W43 或 proj-14
  terminal_seconds bigint NOT NULL DEFAULT 0, -- 贡献：终端实际执行时长
  accepted_tasks   int    NOT NULL DEFAULT 0,
  token_est        bigint NOT NULL DEFAULT 0, -- 终端自报，仅参考
  updated_at       timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (owner_id, period_key)
);
```

优先级

```
-- claim 的 ORDER BY 多一项；fairshare_bonus 由离线任务每 5 分钟刷新
ORDER BY
  CASE t.kind WHEN 'integration' THEN 2 WHEN 'repair' THEN 3 ELSE 1 END,
  (t.priority + o.fairshare_bonus) DESC,
  t.created_at
```

建议算法（区间 0–30，不改变 kind 的先后）：

```
ratio = 该 owner 本周期消耗 / 该 owner 本周期贡献
bonus = round( 30 * (1 - min(ratio, 3) / 3) )
```

`ratio = 0` 时满额 +30；`ratio ≥ 3` 时为 0（不再加成，但不惩罚）。

明确不做

不做	原因
积分 / 代币	需要结算与信任背书，滑向 marketplace；可交易必然出现套利
竞价 / 拍卖	平台无法验证真实 token 消耗，成本必被虚报
排行榜	诱导抢简单任务、拆任务刷数量，直接污染调度与质量信号
给 Agent 做 reward shaping	违反"平台不参与模型行为"的边界，且不可控

需要可见性时，用绝对值不用名次：「本需求由 5 个终端、3 位贡献者完成，A 提供 62% 的算力」。

什么时候才启用

三个条件满足任一：

1. 出现 ≥3 个独立所有者（按"谁掏钱、谁受益"算，不是终端数）；
2. 算力不对等且协商不动；
3. 跨项目抢同一批终端。

在此之前（M0–M2）不实现：单人场景下这层记账纯属负债。

11. 协议

> OpenVow 设计文档集 | v1.1 | 覆盖 §11
> 上一章：调度与配额 | 下一章：安全与审计 | 索引

11.1 端点总表

方法	路径	scope	说明
POST	`/v1/claim`	worker/planner	原子认领（含写集互斥）
POST	`/v1/heartbeat`	worker/planner	续租
POST	`/v1/checkpoint`	worker/planner	中间产物
POST	`/v1/submit`	worker/planner	提交结果与证据
PUT	`/v1/artifacts/{sha256}`	worker/planner	上传二进制
GET	`/v1/artifacts/{sha256}`	worker/planner	下载（含上游产物）
GET	`/v1/tasks/peek`	any	只看不拿
POST	`/v1/graphs`	planner	提交图草案
GET	`/v1/graphs/{id}`	any	读图
POST	`/v1/graphs/{id}/validate`	planner	重跑 V1–V8
POST	`/v1/graphs/{id}/approve`	**human**	批准（生成新 version）
POST	`/v1/graphs/{id}/reject`	**human**	驳回 + reason
POST	`/v1/terminals`	admin	发 token
DELETE	`/v1/terminals/{id}`	admin	吊销
GET	`/healthz`	none	存活探针

权限写死在表里：planner 拿不到 approve；执行终端拿不到 approve；集成资格由 `kind` 决定，不由 token 决定。

11.2 关键报文

claim → 200

```
{
  "task_id": "N-03-7f21",
  "kind": "implement",
  "lease_id": "L-9c2e",
  "lease_expires_at": "2026-10-21T21:10:00Z",
  "attempt": 1,
  "repo_url": "ssh://openvow/proj-14.git",
  "base_commit": "9c1f3ab",
  "writes": ["src/parser/impl/**"],
  "reads": ["src/parser/mod.rs", "tests/parser/**"],
  "frozen": [
    { "path": "src/parser/mod.rs", "sha256": "…" },
    { "path": "tests/parser/basic.rs", "sha256": "…" }
  ],
  "acceptance": { "kind": "green_tests", "run": "cargo test --locked parser" },
  "audit_rate": 0.2,
  "resume": null
}
```

submit → 200

```
{
  "task_id": "N-03-7f21",
  "lease_id": "L-9c2e",
  "attempt": 1,
  "base_commit": "9c1f3ab",
  "files": [{ "path": "src/parser/impl/mod.rs", "sha256": "…" }],
  "evidence": {
    "exit_code": 0, "tests_total": 24, "tests_passed": 24,
    "log_snippet": "…", "duration_s": 214
  },
  "env": { "harness": "claude-code/2.x", "model_family": "…", "prompt_hash": "…" }
}
```

`evidence` 是声明，不是结论；结论来自 L3 的复跑与退出码。

11.3 错误码

码	HTTP	含义	终端应做
`lease_expired`	409	租约被回收	停手，丢弃本地改动
`write_conflict`	409	写集冲突	换任务
`deps_unmet`	409	前驱未 accepted	换任务

码	HTTP	含义	终端应做
`frozen_file_modified`	422	动了冻结文件	还原后重提
`path_outside_writes`	422	提交了写集外文件	删除后重提
`acceptance_failed`	422	L3 未通过	修正后重试
`schema_invalid`	422	L0–L2 失败（带 violations）	修正后重提
`rate_limited`	429	并发/频率超限	退避

11.4 接入方式：MCP 适配器优先

Claude Code、DSH 都是批处理式 CLI：给一段提示，跑完退出，**不会每 5 分钟调 heartbeat。**所以接入不是"写一段 System Prompt"，而是：

形态	适用	说明
MCP server（首选）	Claude Code 等支持 MCP 的 Harness	暴露 5 个工具，agent 在循环里自行调用
CLI wrapper	只接受命令行的 Harness	`openvow run -- <harness>`，wrapper 负责续租、收产物、提交
原生 SDK	自研 Agent	直接调 HTTP

```
openvow_claim(kinds) -> {task, lease, repo, frozen[], resume}
openvow_heartbeat(lease_id) -> {remaining_seconds}
openvow_checkpoint(lease_id, files[], note) -> {checkpoint_id}
openvow_submit(lease_id, files[], evidence) -> {accepted, violations[]}
```

> 协议容易抄，适配器不容易。适配器才是产品的一部分。

12. 安全

> OpenVow 设计文档集 | v1.1 | 覆盖 §12
> 上一章：协议与接入 | 下一章：成本与风险 | 索引

12.1 三种凭证

凭证	持有者	用途	是否违反"不持模型凭证"
模型 API Key	终端	调模型	不涉及，平台永不见
终端 token	平台签发	调 OpenVow	不违反（不是模型凭证）
提交签名（可选）	终端私钥	证明来源	不违反

平台必须持有终端 token，否则任务池可被任意投毒。

12.2 集成容器威胁模型

在容器里跑终端提交的 `build.rs` / `npm install` / `Makefile`，等于让陌生代码在你机器上执行。规格缺一不可：

项	规格
网络	`--network=none`；需装依赖时走只读包代理（白名单域名）
文件系统	rootfs 只读；工作区可写；`tmpfs` 做 home 与临时目录
用户	非 root（uid 1000）；`--cap-drop=ALL`
隔离	seccomp 默认配置；禁止挂载 docker.sock；禁止 privileged
资源	2 CPU / 4GB / 60 分钟 / 磁盘 5GB
宿主	理想为独立 runner 节点，与数据库网络隔离
镜像	固定 digest，写入日志便于复现

`npm install` 的 postinstall 脚本本身就是任意代码执行，包代理只能放行下载，不代理任意域名。

12.3 审计

每个产物必须回答五个问题：谁、何时、基于什么、用什么模型、通过了什么。

字段	来源	可信度
`terminal_id`	平台（token 推导）	高

字段	来源	可信度
`task_id / attempt`	平台	高
`base_commit`	平台	高
`harness / model_family / prompt_hash`	终端自报	低，标 `unverified`
`acceptance_result`	平台复跑或终端声明	分档：`replayed` / `self_reported`

成本与风险

> OpenVow 设计文档集 | v1.1 | 覆盖 §13、§14
> 上一章：安全与审计 | 下一章：路线图与实现基线 | 索引

13. 成本模型

成本项	量级	影响因素
拆解（每项目一次）	150k–400k tokens	仓库规模、需求清晰度
接口 + 测试（每模块）	40k–120k tokens	模块复杂度；**3.0 新增成本**
实现（每任务）	20k–80k tokens	上下文、重试次数
平台复跑	1–10 分钟 CPU/次	信任分
集成全量	5–60 分钟 CPU/次	项目规模、测试套件
人工评审	5–15 分钟/任务	图审批 + 抽样

接口 + 测试约占单模块总 token 的 25–35%。 这是有意的交换：

> 用 30% 的前置成本，换掉集成阶段全盘返工的风险。分布式开发里，返工成本是前置成本的 5–10 倍。

连这三成都不愿意付，那就别并行。

14. 风险登记册

#	风险	触发信号	对策	残余
R1	切分粒度不当	单任务 diff 中位数 < 50 或 > 600 行	V5 + 人工审批	中
R2	并行度低（热文件）	图的理论并行度 < 2	三策略（§10.4），否则换项目	高（结构限制）
R3	验收造假	复跑失败率 > 5%	冻结 + 红测试 + 信任分	低
R4	集成反复失败	同 group repair ≥ 2 次	升级 needs_human，重拆子图	中
R5	评审疲劳	人工评审 > 20 分钟/任务	只审 diff + 抽检	中
R6	环境差异假失败	复跑失败但本地绿	固定镜像 digest	中
R7	供应商风控	终端账号异常	终端侧限速；平台不参与	中（不可控）
R8	平台成瓶颈	单库 QPS > 50	只读副本 + 缓存	低

#	风险	触发信号	对策	残余
R9	许可证污染	出现 GPL 片段而项目是 MIT	SCA 纳入 L2	中
R10	成本失控	单任务 token > 200k	看板告警 + 粒度回退	中

路线图与实现基线

> OpenVow 设计文档集 | v1.1 | 覆盖 §15、§16
> 上一章：成本与风险 | 下一章：决策与未决 | 索引

---

15. 路线图

M0 • 单终端闭环（1–2 周）

范围：手工建任务、CLI、单终端、无图、无并行。类型 `interface → test → implement` 手工串。

验收：

1. 三个任务串起来，最后一个 `accepted`；
2. 终端改冻结测试 → `frozen_file_modified`；
3. 空洞测试 → 红测试验收失败；
4. 越界写文件 → `path_outside_writes`；
5. 产物可按 sha256 回溯到 terminal + 时间 + attempt。

> **M0 不并行，但它已经是完整的产品**：契约 + 验收 + 冻结 + 归集。后面所有东西都是在它之上加并发。

M1 • 图与契约（2–3 周）

`decompose` + planner + V1–V8 + 人审。验收：5 节点图能过校验并派发；写集冲突、接口缺失、类型拓扑错误各能被拦；驳回后 planner 能带 reason 重拆。

M2 • 并行与信任（2 周）

多终端、写集互斥 SQL、租约/心跳/检查点、信任分、token 吊销。验收：10 终端并发 10 任务，0 重复、0 写集相交；`kill -9` 后 60s 内回队列且 attempts+1；检查点可续跑。

M3 • 集成与修复（2–3 周）

集成任务自动物化、容器全量验证、失败包、repair 环路、canonical 合并。验收：故意造语义冲突 → 集成失败 → 自动 repair → 修复后合并；连续 3 次失败升级 `needs_human`。

M4 • 规模化（按需）

两级拆解、merge group 分片、看板、许可证扫描、成本统计。

16. 实现基线（M0）

本节是 §15 中 M0 的实现约定；逐行细节见《M0 施工清单》。

16.1 技术选型

项	选择	理由
语言	Go 1.22+	单二进制、标准库路由、无 CGO 依赖
存储	SQLite（modernc，纯 Go）	单文件、免安装；M2 迁 Postgres 时语义不变
路由	`net/http`	6 个端点不值得上框架
哈希	`crypto/sha256`	内容寻址

16.2 提交处理流水线

```
submit
  ├─ 0. 幂等：submissions(task_id, attempt) 已存在 → 返回上次结果
  ├─ 1. 租约：lease_id 匹配 且未过期
  ├─ 2. L0：路径规范化、⊆ writes、大小上限
  ├─ 3. L1：重算 sha256、冻结文件未被改动
  ├─ 4. 落盘：复制 repo → 用冻结版本覆盖 → 应用提交文件
  ├─ 5. L3：按 acceptance.kind 执行 build / run
  └─ 6. 写事件、更新状态
```

第 4 步是防作弊的核心：不管终端交了什么测试，一律用 `frozen` 的版本覆盖，再跑验收。

16.3 CLI

```
openvow serve --db ./openvow.db --repo ./demo --addr 127.0.0.1:7711
openvow terminal add --name t1 --out ./t1.token
openvow task new --kind test --title "解析器测试" --after T-0001 \
  --writes "src/parser/parser_test.go" \
  --acceptance '{"kind":"red_tests","build":"go test -run XXX ./parser","run":"go test ./parser"}'
openvow claim
openvow submit --dir .
```

`--after` 自动继承前驱 id 与冻结文件哈希，这是 M0 最关键的便利。

16.4 安全提醒

M0 在宿主机上执行 `acceptance` 命令，等于执行终端提交的代码。**只在你信任的仓库与终端上运行**，不要部署到共享环境。容器化是 M3 的事。

决策与未决

> OpenVow 设计文档集 | v1.1 | 覆盖 §17、§18
> 上一章：路线图与实现基线 | 下一章：设计问题 | 索引

17. 关键决策（ADR）

#	决策	否决的方案	理由
01	拉模式（终端 claim）	推模式	风控与成本必须留在终端
02	平台不内置任何模型	内置拆解/审查 Agent	一旦内置，边界破
03	Postgres `SKIP LOCKED`	Redis 锁 / Celery / Prefect	一条 SQL 解决，少一个中间件
04	内容寻址产物库	按任务存目录	去重、可回溯、防路径绕过
05	拆解也是任务	平台内置 planner	保持不变量
06	接口前置为独占任务	靠 `decisions[]` 口头约定	文件不相交 ≠ 语义能对接
07	测试独立 + 红测试 + 冻结	实现方自带测试	否则验收可被自己判卷
08	集成是任务	作为流水线阶段	失败需要工作载体
09	MCP 适配器优先	一段 System Prompt	批处理 CLI 无法定时心跳
10	匿名只在协议层	真匿名	私有项目需要追责与治理
11	风控叙事降级	以"防封号"为核心卖点	套利不可持续
12	校验四层，合并是任务	L0–L4 五层	合并需要判断，不能机械化
13	平台只决定顺序，不改字节	平台自动修复	改字节破坏"零二次加工"
14	不引入奖励 / 代币 / 竞价，采用配额与优先级	积分制、竞价、排行榜、Agent reward shaping	私有场景的痛点是分配不是激励；可交易积分必然被刷；平台无法验证真实 token 消耗

18. 未决问题

1. **merge group 怎么切？** 默认整图；大图按模块拆多个 group 并行集成，但跨 group 的接口一致性会退化为"组间集成"。需实测阈值。
2. **红测试的 `min_failing_ratio` 定多少？** 0.8 是拍的，有些模块天然一半是类型测试。
3. **repair 的写集要不要放开到全仓？** 放开能修得动，但破坏"写集可控"的直觉。倾向默认放开 + 人工过目。
4. **信任分会不会惩罚慢工出细活？** 一次通过率奖励"稳"，可能抑制"敢重构"。必要时加返工率作第二维。

5. **是否支持从 issue tracker 直接建需求？** 降低门槛，但会把"需求条目化"外包给外部系统。
6. **配额的周期与权重怎么定？** 按周还是按项目结算；`terminal_seconds` 之外要不要把 token 折算进来（自报的数不可信，可能只作为参考列展示）。等真正出现多所有者时用真实数据再定。

19. 设计问题清单

> OpenVow 设计文档集 | v1.1 | 覆盖 §19
> 上一章：决策与未决 | 下一章：验证实验 | 索引

> 这一章存在的理由：**文档的章节完整，不等于问题的完整。** 前 18 章里有不少条目，其实是"用假设回答的问题"，被写成了决策的样子。本章把它们拆开，标出哪些站得住、哪些只是假设、哪些根本没有答案。

19.1 读法

状态	含义
已定	有明确答案，且能说出为什么否掉其他选项
假设	有答案，但没有证据；一旦不成立，相关章节失效
未决	没有答案。多数是当前信息不足，少数是刻意留空

全章共 34 问，其中 8 个没有答案，集中在 §19.6。

19.2 L1 命题层：这件事成立吗

#	问题	现状	状态
1	真正的瓶颈是墙钟、评审，还是质量?	默认按"墙钟"设计。若瓶颈是评审，并行是负收益	假设
2	有多少需求能被切成文件不相交的块?	假设"以新增模块为主"，没测过比例	假设
3	有多少子任务能写出可执行的验收?	红测试默认都能写；UI、体验、架构类写不出	未决
4	比"一个人 + 一个 Harness"快多少?	没有任何基准实验	未决

19.3 L2 边界层：管到哪

#	问题	现状	状态
5	平台能持有哪些凭证?	终端 token 可以，模型 Key 不行	已定
6	平台能不能替人做架构决策?	不能，`decisions[]` 由人给	已定
7	匿名到什么程度?	协议层匿名、平台层实名	已定
8	产物归属与许可证谁负责?	只提了 SCA，没有责任主体	未决

#	问题	现状	状态
9	平台挂了终端能继续干吗?	不能，没有离线模式	假设
10	需求变更到什么频率，体系失效?	有图版本机制，没有频率阈值	未决

19.4 L3 机制层：内部怎么转

#	问题	现状	状态
11	任务切多细?	≤ 4 文件 / ≤ 400 行，是拍的	假设
12	为什么接口必须先冻结?	语义冲突无法自动发现	已定
13	探索性任务（调研 / 原型 / 调参）放哪?	没有位置	未决
14	测试由谁写?	独立任务 + 红测试验收	已定
15	测试写错了需求怎么办?	只能靠覆盖度 + 抽检	假设
16	不可测试的需求怎么办?	无解，退回人审	未决
17	签名级校验够不够?	不够，靠编译兜底	假设
18	热文件怎么处理?	三策略，优先级明确	已定
19	并行度的真实上限由什么决定?	三者取小：写集 × 接口 × 评审。没量化	假设
20	长任务失败怎么办?	检查点，但终端是否配合未知	假设
21	异构终端能力差异怎么对齐?	按 kind 分派，缺能力矩阵	未决
22	合并冲突的本质是什么?	文本冲突已解，语义冲突未解	假设
23	合并顺序可交换吗?	假设可交换，依赖边可能不足	未决
24	谁修集成失败?	repair 任务，换终端	已定
25	repair 放开写集会破坏隔离吗?	会，靠人工过目兜底	假设
26	一次通过率代表质量吗?	只代表"可验收性"	假设
27	评审带宽不够时系统怎么做?	没有背压机制，只会继续生产	未决
28	平台怎么防作弊?	四层矩阵	已定
29	复跑成本怎么控?	信任分抽检	已定
30	终端能刷信任分吗?	能：先在小任务上稳定表现，再在大任务上冒险	未决
31	需求条目谁写、什么格式、多粗?	只说"人写"，没有格式与粒度标准	未决
32	一张图审批要多久?	没测。它直接等于系统吞吐上限	未决

19.5 L4 治理层：长期怎么活

#	问题	现状	状态
33	技术债谁负责?	无人。L3 只管"能跑"，不管"能改"	未决
34	代码风格 / 架构漂移怎么办?	linter 只管格式	假设
35	知识怎么沉淀?	图历史 + amendments，没有检索设计	未决
36	多所有者怎么分配?	fair-share 已设计、未启用	已定
37	要不要经济激励?	不要	已定
38	指标怎么采?	六个指标，没有采集方案	假设
39	新校验规则怎么灰度?	未设计，直接上线会误伤在途任务	未决
40	数据怎么迁移?	schema 版本策略未定	未决

19.6 八个没有答案的

（1）可验收性覆盖率（#3）

红测试是整套设计的地基，但它只适用于"行为可断言"的任务。真实需求里通常有三类工作：能写断言的（解析、计算、协议）、只能看结果的（UI 布局、CLI 手感）、只能做判断的（架构、选型、命名）。

如果可验收任务只占四成，这套系统就只能覆盖四成工作量，剩下仍要人做——并行收益按四成折算，回本门槛立刻翻倍。当前没有任何数据。

最便宜的验证：拿一个真实需求，让三个人各切一遍，统计"能写出红测试的任务"占比（§20 实验 A）。

（2）并行度的真实上限（#19）

写集不相交只是必要条件。真实并行度 = min(写集自由度, 接口依赖深度, 评审带宽)。

接口先行会拉长关键路径：`interface → test → implement` 是天然串行的链，只有多条链之间才能并行。一个 6 模块的项目，可能是 6 条串行链并行，而不是 18 个任务并行。

这意味着"任务数"和"并行度"没有关系，而现有的指标体系一直在数任务。

（3）探索性任务没有位置（#13）

"先调研三个方案再定接口"——这类任务写不出 `acceptance`，也不该有 `writes`（产出是认知，不是文件）。现在的类型学里它是非法的，而真实项目开头约两成时间都花在这上面。

要么加一个 `explore` 类型（产出是决策文档，验收是"人读完点头"），要么明确说"不在范围内，你自己做"。现在的状态是假装它不存在。

（4）评审背压缺失（#27）

系统会匀速生产，但评审速度是人的生理限制。没有背压，结果就是队列无限增长。

需要一条机制：当未评审的 `accepted` 任务超过 N 个时，整体降低 claim 优先级，而不是继续灌。这是"生产五倍代码、积压五倍"这个风险唯一的技术出口。

（5）技术债无人负责（#33）

L3 通过 = 行为符合断言。一个把十个函数塞进一个文件、复制粘贴三遍的实现，能百分之百通过。三个月后没人敢改。

这不是"再加个 linter"能解决的，它需要一个**角色**：架构守门人。设计里没有这个角色，只有"人"这一个笼统主体。要么加角色，要么明确接受"这套系统会稳定地生产技术债"。

（6）需求变更频率阈值（#10）

图版本、stale、rebase 都设计了，但没回答：变更多频繁时，这套系统比串行还慢。改一次图 = 重新审批 + 一批任务 stale + 重做。若一周改三次需求，开销会吃掉全部并行收益。

（7）信任分可被策略性刷取（#30）

trust 由"近 20 次一次通过率"决定，复跑比例由 trust 决定。终端完全可以：接 20 个简单任务把 trust 刷到 0.95，再在第 21 个大任务上作弊，此时复跑率只有 5%。

这是典型的**冷启动 + 对抗性**问题。签名与抽样都不够，需要"按任务风险加权"的抽样——大 diff、改接口、碰热文件的任务永远 100% 复跑，而"风险"本身要能算出来。

（8）模型变强会让这套系统变贵

如果模型能一次性写完整个模块并自己跑通测试，契约成本（接口 + 测试 + 审批）就从"必要投资"变成"纯开销"。

这套设计的价值随"任务粒度 × 并行收益"增长，随"模型单次能力"下降，而后者在涨。所以它有一个**时间窗口**，不是永久成立的架构。

诚实的定位：它的长期价值不在并行，而在**审计、成本分摊、多主体协作的秩序**。若目标只是"一个人更快"，它可能两年后就不值了。

19.7 三个前提

#	前提	不成立的后果	验证方式
A	需求能被切成文件不相交的块	并行度 ≈ 1，整套机制变成纯开销	§20 实验 A
B	相当比例的任务能写出可执行验收	L3 覆盖不了，退回人审，吞吐被评审锁死	§20 实验 A
C	有人愿意在动手前写契约	图永远停在 draft，平台空转	§20 实验 C

三个里 A 和 B 可以用一个下午的实测证伪。若两者都低于红线，正确的做法不是优化系统，而是**收窄定位**——只做"可验收的模块级开发"，其余明确排除。

20. 验证实验

> OpenVow 设计文档集 | v1.1 | 覆盖 §20
> 上一章：设计问题 | 下一章：附录与结语 | 索引

> 原则：**不要做"能证明设计对"的实验，要做"能证伪设计"的实验。** 红线事先写好，结果出来只对照，不改标准。

20.1 六个生命体征

整套设计是否成立，最后会收敛到六个数字上。

代号	名称	定义	红线
P	并行度	按写集冲突 + 依赖做调度模拟，最大在途任务数的平均值	≥ 3
R	可验收率	能写出「基线红 / 实现绿」测试的任务数 ÷ 总任务数	≥ 0.6
C	契约开销	走完整流程的墙钟 ÷ 一人直写的墙钟	≤ 1.5
S	语义冲突率	并行模块合并时出现编译 / 测试失败的比例	≤ 0.2
T	评审吞吐	每天能审完的任务数	≥ 生产速度 × 0.5
V	变更税	需求变更导致重做的工时 ÷ 总工时	≤ 0.2

后续每一个实验，都是为了拿到其中一个数字。

20.2 第一梯队：半天，决定生死

实验 A | 可切分 + 可验收

一次拿 **P** 和 **R**，成本最低、决定力最大。

材料：三个真实需求，不用 demo 仓库——①一个新增功能（行为明确）；②一次存量改造（要动老代码）；③一组 bug 修复（5–10 个相关缺陷）。

做法：

1. 每个需求切成任务，记录 `writes` 与 `depends_on`；
2. **换一个人**（或另一个模型）逐任务判断：能不能写出「在基线上失败、在正确实现上通过」的测试；
3. 用这 9 张图跑调度模拟（按 `writes` 相交 + 依赖），算 P；
4. 统计 R。

产出：每个需求的 P 与 R，一张表。

红线：三个需求里有两个 P < 2，或平均 R < 0.5。

不成立时：P 低 → 收窄到"新增模块"场景，或在流程前加一个 `refactor` 任务先把存量改造成可并行形态；R 低 → 增加"人审验收"任务类型，把不可验收的工作显式排除，而不是假装它们有验收。

纪律：切分的人和验证的人不能是同一个。自己切自己验，一定得到"看起来都能切"的结论。

实验 B | 热文件分布

材料：三个真实仓库。

做法：对每个仓库问「新增一个模块，要动几个既有文件？」（注册表、`mod.rs`、路由、依赖清单），取中位数。

产出：热文件中位数。

红线：中位数 > 2。

不成立时：把"消除热文件"变成接入前置条件——这是一次性成本，但会改变项目结构，需要先说服人。

20.3 第二梯队：一周，决定机制

实验 C | 契约开销比

材料：一个中等模块（200–400 行）。

做法：同一模块走两条路——路径 a：一个人直接写并跑通测试；路径 b：`interface → test → implement → integration` 完整流程。记录墙钟、token、返工次数、最终代码可读性（找没参与的人盲评）。

产出：C = b 墙钟 ÷ a 墙钟。

红线：C > 1.5 且返工率未降到 50% 以下。

不成立时：把 `interface` 与 `test` 合并成一个任务（一次产出骨架 + 测试），或用模块级模板复用契约，压低边际成本。

这一条最关键：它直接检验"用三成前置成本换掉集成返工"这个核心交易。若 C 是 2.5，那不是设计问题，是假设错了。

实验 D | 红测试可行性

做法：给 10 个任务写测试，只允许"在骨架上必须失败"的写法。统计：①一次通过 `red_tests` 的比例；②**假红灯率**——抽样做变异测试（把骨架里某个函数改成返回错误值，看测试能否抓住）。

产出：两个比例。

红线：假红灯率 > 20%。

不成立时：退回"冻结测试 + 人写测试"模式（成本高但可靠），或把变异测试作为集成前的抽样手段。

实验 E | 语义冲突

材料：两个"文件不相交但接口相关"的模块（如解析器与求值器）。

做法：两个终端并行实现，各自跑通自己的测试后合并。记录是否编译失败、是否测试失败、原因是"接口对不上"还是"行为不一致"。重复 5 组。

产出：S。

红线：S > 0.2。

不成立时：在 `test` 任务里增加"跨模块契约测试"（调用方写的、针对接口的测试），把语义不一致前移到任务内。

20.4 第三梯队：随真实使用产生

实验 F | 信任分对抗

做法：写一个"恶意终端"脚本：先接 20 个小任务全过（刷 trust 到 0.95），第 21 个提交"测试被改软但哈希伪造"的产物，看抽检能否抓住。

产出：抓住率。

红线：抓不住。

不成立时：改按任务风险加权抽检——大 diff、改接口、碰热文件的永远 100% 复跑，与 trust 无关。

实验 G | 评审吞吐

做法：记录每次审图、审产物的实际分钟数。

产出：T。

红线：T < 生产速度的一半。

不成立时：加背压阀（未评审的 accepted 任务超过 N 个时整体降优先级），同时提高自动验收覆盖比例。

实验 H | 变更税

做法：记录真实项目里需求变更次数、每次牵连任务数、重做工时。

产出：V。

红线：V > 0.2。

不成立时：只接"需求稳定"的项目，或把变更窗口前移（图批准后 24 小时内允许免费改）。

实验 I | 反向基准

做法：同一模块，一次让当前最强模型整包做完，一次走完整流程，比总成本（token + 人时）。

产出：成本比。

意义：这条不是看现在，是看趋势。若整包做已经接近，这套设计的时间窗口就在关闭——那时它的价值应收缩到"审计与协作"，而不是"并行"。

20.5 实验纪律

1. **材料必须真实。** 用自己写的 demo 仓库做实验，结论一定是"很适合切分"——因为你就是照着设计写的。
2. **红线先写。** 看完数据再定标准，等于没有标准。
3. **记录失败原因，不只记录成功。** 每次切分失败、每次红测试写不出来，都是设计约束的真实来源。
4. **三类需求都要**：新功能、存量改造、bug 群。只在新增模块上成立，等于只覆盖三分之一的真实工作。
5. **不要优化实验。** 第一个实验的目的是拿到数字，不是做出一个好用的原型。

20.6 如果只做一个

做实验 A + B，一个下午，得到 P 和 R。

- P ≥ 3 且 R ≥ 0.6 → 继续做 M0，同时用实测值替换本文里那些拍脑袋的阈值；
- P < 2 → 收窄定位到"新增模块开发"，把存量改造明确排除；
- R < 0.5 → 承认"大部分工作没有可执行验收"，重新设计人审环节，别用红测试假装覆盖。

这两个数字一旦拿到，本文里六成的争论会自动消失——因为它们本来就是在争"这个前提成不成立"。

20.7 结果回写：实验替换哪些常量

实验的意义不是"验证一下"，而是**把假设换成数字**。下表说明每个数字应该回写到本文的哪一处。

实验	产出	回写位置	现在那里的值
A	P	§14 R2、§15 M2 验收	"并行度 < 2 告警"（拍的）
A	R	§9.2 L3 覆盖、§3.4 回本门槛	无覆盖率要求
B	热文件中位数	§10.4 策略优先级、§3.2 scope guard	无阈值
C	C	§13 成本模型、§17 ADR-06/07	"25–35% 前置成本"（估的）
D	假红灯率	§7.3 红测试判定	`min_failing_ratio = 0.8`（拍的）
E	S	§7.3 跨模块契约测试	未设计
F	抓住率	§10.3 信任分抽检	5% / 20% / 50%（拍的）
G	T	§10 背压（待补）	无背压
H	V	§3.4 回本门槛	"每周 15 任务"（拍的）
I	成本比	§3.5 定位、§19.6(8)	无数据

> 一条元规则：**任何没有实验来源的数字，在评审时都应被视为"待定"，而不是"已定"。**

附录与结语

> OpenVow 设计文档集 | v1.1 | 覆盖 §21、§22
> 上一章：验证实验 | 索引

---

21. 附录

A. 错误码全集

域	码	HTTP
租约	`lease_expired`	409
租约	`lease_invalid`	409
调度	`write_conflict`	409
调度	`deps_unmet`	409
调度	`kind_not_allowed`	403
契约	`path_outside_writes`	422
契约	`frozen_file_modified`	422
契约	`path_invalid`	422
产物	`artifact_missing`	422
产物	`artifact_too_large`	413
验收	`acceptance_failed`	422
验收	`acceptance_timeout`	422
图	`graph_schema_invalid`	422
图	`graph_not_validated`	409
图	`graph_not_approved`	409
通用	`rate_limited`	429
通用	`unauthorized`	401
通用	`forbidden`	403

B. 配置项

键	默认	说明
`claim.lease_seconds`	1800	租约时长
`claim.max_attempts`	2	认领重试上限
`claim.poll_interval_hint`	30s	建议轮询间隔
`artifact.max_file_bytes`	2097152	单文件上限
`artifact.max_total_bytes`	52428800	单次提交上限
`acceptance.default_timeout`	900	验收超时
`integration.timeout_seconds`	3600	集成超时
`integration.max_attempts`	3	集成重试上限
`graph.max_nodes`	12	单图节点上限
`task.max_files`	4	单任务文件数上限
`task.max_diff_lines`	400	单任务 diff 上限
`trust.min_samples`	5	信任分起算样本
`trust.replay_rate`	0.05	高信任终端复跑比例

C. 性能基线（目标值，非实测）

场景	目标
`claim`（10 万 queued，200 并发）	p99 ≤ 20ms
`submit` L0–L2（50 文件）	≤ 200ms
产物上传（10MB）	≥ 20MB/s
单实例日吞吐	≥ 5 万次提交
库容量	100 万任务以内单库

D. 参考

- PostgreSQL `FOR UPDATE SKIP LOCKED` 语义
- Model Context Protocol 工具定义规范
- SLSA 构建隔离基线（容器规格参考）

22. 结语

1.0 想的是"把活分出去"，2.x 补的是"收回来能用"，3.0 补的是"怎么保证分出去的时候接口已经说清楚、判卷的人不是考生、合并失败有人接活"。

分布式开发真正的难点从来不是并发，是**契约**和**责任的落点**。这份设计里，每一件产出内容的事都有人负责，而且负责的人不是同一个人。

OpenVow M0 施工清单

> 本文是见 13-路线图与实现基线.md §16「实现基线」的逐行细化版。 **范围**：单终端、无图、无并行、手工串链。**目标**：两周内跑通一个"能判卷、判卷的人不是考生"的最小闭环。

---

1. M0 交付什么

一个 Go 单二进制 `openvow`，同时是**服务端**和**终端 CLI**；一个 SQLite 库；一个能验证五条验收标准的脚本。

跑通后你会得到：

```
openvow task new --kind interface ...   → 终端 claim → 写骨架 → submit → accepted
openvow task new --kind test --after T1 → claim → 写测试（必须在骨架上红） → accepted
openvow task new --kind implement --after T2 → claim → 实现 → 测试绿 → accepted
```

M0 不并行，但它是完整产品：契约、冻结、判卷、归集、追溯全在。后面所有并发能力都是在它之上加。

---

2. 技术选型（定死）

项	选择	理由
语言	**Go 1.22+**	单二进制、部署零依赖、标准库自带路由（`http.ServeMux` 的 method 模式）
存储	**SQLite**（`modernc.org/sqlite`，纯 Go 无 CGO）	单文件、免安装；M2 迁移 Postgres 时字段语义不变
路由	标准库 `net/http`	6 个端点不值得上框架
哈希	`crypto/sha256`	内容寻址
配置	环境变量 + 命令行 flag	不上配置文件
测试	标准库 `testing` + `httptest`	无第三方

为什么不是 Python：M0 的核心是"文件落盘 + 哈希 + 起进程跑命令 + 事务"，Go 在这四件事上都是标准库直出；Python 会引入虚拟环境、依赖锁定、启动器三个额外问题。如果团队只会 Python，替换成本主要在 §6 的四个 handler，表结构与验收语义完全不变。

为什么不用 Postgres：M0 没有并发认领，`SKIP LOCKED` 用不上。但 DDL 要写成能平滑迁移的样子（见 §4 的注释）。

---

3. 仓库结构

```
openvow/
├── cmd/openvow/main.go          # 子命令入口：serve / terminal / task / claim / submit / peek
├── internal/
│   ├── api/                     # HTTP handler (6 个)
│   │   ├── claim.go
│   │   ├── heartbeat.go
│   │   ├── submit.go
│   │   ├── artifacts.go
│   │   ├── tasks.go             # peek
│   │   └── auth.go              # token → terminal
│   ├── store/                   # SQLite：迁移、查询、事务
│   │   ├── migrate.go
│   │   ├── tasks.go
│   │   ├── artifacts.go
│   │   └── events.go
│   ├── contract/                # 契约解析与校验
│   │   ├── paths.go             # 规范化 + glob 匹配
│   │   ├── freeze.go            # 冻结文件比对
│   │   └── acceptance.go        # 四种验收的执行器
│   ├── runner/                  # 在工作区执行命令（带超时、捕获输出）
│   └── workspace/               # 复制 repo、应用文件、清理
├── scripts/
│   └── m0-acceptance.ps1        # 五条验收标准的自动化验证
├── testdata/demo/               # 一个 30 行的 Go 小项目，用于端到端
└── README.md
```

---

4. 数据模型（M0 DDL）

```
PRAGMA journal_mode = WAL;
PRAGMA foreign_keys = ON;

CREATE TABLE terminals (
  id          TEXT PRIMARY KEY,
  name        TEXT NOT NULL UNIQUE,
  token_hash  BLOB NOT NULL,          -- sha256(token)
  created_at  TEXT NOT NULL,
  banned_at   TEXT
);

CREATE TABLE tasks (
  id             TEXT PRIMARY KEY,
  kind           TEXT NOT NULL CHECK (kind IN ('interface','test','implement')),
  title          TEXT NOT NULL,
  summary        TEXT NOT NULL DEFAULT '',
  -- 迁移预留：M2 换成 Postgres text[]，语义不变
  writes_json    TEXT NOT NULL,       -- ["src/parser/**"]
  reads_json     TEXT NOT NULL,       -- ["src/parser/mod.go"]
  frozen_json    TEXT NOT NULL,       -- [{"path":"a.go","sha256":"…"}]
  acceptance_json TEXT NOT NULL,      -- 见 §5.2
  requires_json  TEXT NOT NULL,       -- ["T-0001"] 全部 accepted 才可 claim
  state          TEXT NOT NULL,       -- queued/claimed/validating/accepted/rejected/needs_human
  attempts       INTEGER NOT NULL DEFAULT 0,
  max_attempts   INTEGER NOT NULL DEFAULT 2,
  lease_id       TEXT,
  lease_until    TEXT,                -- RFC3339 UTC，服务端时钟
  terminal_id    TEXT REFERENCES terminals(id),
  created_at     TEXT NOT NULL,
  updated_at     TEXT NOT NULL
);

CREATE TABLE artifacts (
  sha256     TEXT PRIMARY KEY,
  size       INTEGER NOT NULL,
  created_at TEXT NOT NULL
);

CREATE TABLE task_files (
  task_id  TEXT NOT NULL,
  attempt  INTEGER NOT NULL,
  path     TEXT NOT NULL,             -- 相对仓库根，已规范化
  sha256   TEXT NOT NULL REFERENCES artifacts(sha256),
  PRIMARY KEY (task_id, attempt, path)
```

```
);

CREATE TABLE submissions (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  task_id     TEXT NOT NULL,
  attempt     INTEGER NOT NULL,
  lease_id    TEXT NOT NULL,
  accepted    INTEGER NOT NULL,       -- 0/1
  code        TEXT,                   -- 拒绝原因码
  detail_json TEXT,                   -- violations / 日志摘要
  created_at  TEXT NOT NULL,
  UNIQUE (task_id, attempt)           -- 幂等键
);

CREATE TABLE events (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  task_id      TEXT,
  kind         TEXT NOT NULL,         -- claim/heartbeat/submit/reject/expire
  payload_json TEXT,
  at           TEXT NOT NULL
);
```

迁移注意：`writes_json` 现在用 JSON 文本，因为 M0 不需要数组交叠运算。M2 换 Postgres 时改成 `text[]` 并加 GIN 索引，`&&` 才能用上。

---

5. 契约

5.1 任务创建参数

```
openvow task new \
  --kind implement \
  --title "解析器主流程" \
  --writes "src/parser/impl/**" \
  --reads "src/parser/mod.go,tests/parser/**" \
  --after T-0002 \
  --acceptance '{"kind":"green_tests","build":"go build ./...","run":"go test ./parser","timeout_s":300}'
```

`--after` 做三件事（这是 M0 最关键的便利）：

1. `requires` 追加该任务 id；
2. 若是 `implement` 且前置是 `test`：把该 test 任务**全部产物路径与哈希**写入 `frozen_json`；
3. 若是 `test` 且前置是 `interface`：把接口文件写入 `frozen_json`。

5.2 三种验收

```
// interface
{"kind":"skeleton_build","build":"go build ./...","timeout_s":300}

// test (必须在骨架上红)
{"kind":"red_tests","build":"go test -run XXX ./parser","run":"go test ./parser",
 "expect":"fail","min_failing_ratio":0.8,"timeout_s":300}

// implement
{"kind":"green_tests","build":"go build ./...","run":"go test ./parser","timeout_s":300}
```

判定矩阵：

kind	build	run	结论
`skeleton_build`	0	不跑	0 → accepted
`red_tests`	0	≠0 且失败率 ≥ 0.8	accepted
`red_tests`	0	0（全绿）	`tests_not_red` —— 这是空测试的信号
`red_tests`	≠0	—	`build_failed`
`green_tests`	0	0	accepted
`green_tests`	0	≠0	`acceptance_failed`

---

6. HTTP 端点（6 个）

方法	路径	请求	响应
POST	`/v1/claim`	`{"kinds":["implement"],"labels":["go"]}`	200 任务 / 204 无活
POST	`/v1/heartbeat`	`{"task_id":"…","lease_id":"…"}`	`{"remaining_seconds":n}`
POST	`/v1/submit`	见下	`{"accepted":bool,"code":"…","violations":[]}`
PUT	`/v1/artifacts/{sha256}`	原始字节	204
GET	`/v1/artifacts/{sha256}`	—	原始字节
GET	`/v1/tasks/peek?state=queued`	—	任务列表

鉴权：`Authorization: Bearer <token>`，中间件换成 `terminal_id` 注入 context。

submit 报文：

```
{
  "task_id": "T-0003",
  "lease_id": "L-7f3a",
  "attempt": 1,
  "files": [{"path": "src/parser/impl/mod.go", "sha256": "…"}],
  "evidence": {"exit_code": 0, "duration_s": 42, "log_snippet": "…"}
}
```

---

7. 提交处理流水线（核心逻辑，按顺序）

```
submit
  ├─ 0. 幂等：submissions(task_id, attempt) 已存在 → 直接返回上次结果
  ├─ 1. 租约：lease_id 匹配 且 lease_until > now，否则 409 lease_expired
  ├─ 2. L0 结构
  │    ├─ 路径规范化（拒绝 .. / 绝对路径 / 符号链接）
  │    ├─ 每个路径必须命中 writes 的某个 glob，否则 path_outside_writes
  │    └─ 单文件 ≤ 2MB，总计 ≤ 50MB
  ├─ 3. L1 完整
  │    ├─ 重算 sha256，与提交值比对
  │    ├─ 命中 frozen 且哈希不同 → frozen_file_modified
  │    └─ frozen 文件必须**未被提交**（提交了也算改）
  ├─ 4. 落盘 workspace
  │    ├─ 复制 repo 到临时目录
  │    ├─ 用 frozen 的版本覆盖（防止终端夹带）
  │    └─ 应用提交的文件
  ├─ 5. L3 行为：按 acceptance.kind 执行 build / run，带超时
  ├─ 6. 写事件、更新状态
  │    ├─ accepted：任务置 accepted，解锁下游
  │    └─ 其他：attempts+1；超限 → needs_human，否则回 queued
  └─ 返回
```

第 4 步是防作弊的核心：不管终端交了什么测试文件，一律用 `frozen_json` 里的版本覆盖。终端改测试 = 无效，且会被第 3 步判 `frozen_file_modified`。

---

8. CLI 命令

```
# 起服务（repo 是被拆解的目标工程目录）
openvow serve --db ./openvow.db --repo ./demo --addr 127.0.0.1:7711

# 终端注册（输出一次性的 token）
openvow terminal add --name t1 --out ./t1.token

# 建任务
openvow task new --kind interface --title "解析器骨架" --writes "src/parser/**" \
  --acceptance '{"kind":"skeleton_build","build":"go build ./..."}'
openvow task new --kind test --title "解析器测试" --after T-0001 \
  --writes "src/parser/parser_test.go" \
  --acceptance '{"kind":"red_tests","build":"go test -run XXX ./parser","run":"go test ./parser"}'
openvow task new --kind implement --title "解析器实现" --after T-0002 \
  --writes "src/parser/impl/**" \
  --acceptance '{"kind":"green_tests","build":"go build ./...","run":"go test ./parser"}'

# 查看
openvow task list
openvow task show T-0002

# 终端侧
export OPENVOW_TOKEN=$(cat t1.token)
openvow claim                     # 打印当前任务 + 契约 + frozen 列表
openvow submit --dir .            # 扫描 writes 命中的文件，算哈希，提交
openvow heartbeat --lease L-7f3a  # 长任务手动续租（M0 无自动心跳）
```

`openvow claim` 的输出要**直接可读**，因为 M0 的"适配器"就是人 + Claude Code：把人看到的这段贴进 CLI 会话，Agent 就知道该干什么、不许碰哪些文件。

---

9. 子任务分解（自己吃自己的狗粮）

#	任务	产出	验收
S1	存储层	`store` 包 + 迁移 + 种子数据	表能建、事务能回滚、`go test ./internal/store` 绿
S2	契约层	路径规范化、glob 匹配、冻结比对	表格驱动的 20 个用例全绿（含 `..` 绕过、绝对路径、符号链接）
S3	提交流水线	§7 的 6 步	6 个拒绝码各有单测，顺序正确（幂等先于租约）
S4	HTTP + CLI	6 端点 + 6 子命令	`curl` 全覆盖；CLI 端到端能 claim→submit
S5	验收脚本	`m0-acceptance.ps1`	五条标准全过，输出可读报告

建议顺序：S1 → S2 → S3 → S4 → S5。S2 可以并行做（无依赖），但 M0 只有你一个人，串行更省心。

---

10. 五条验收标准（Definition of Done）

脚本一跑，输出五行 PASS/FAIL：

#	标准	验证方式
1	三个任务串起来，最后一个 `accepted`	串行 claim→submit 三次，断言最终状态
2	改冻结测试被拒	提交前改 `parser_test.go` 一个字节 → 期望 `frozen_file_modified`
3	空测试被拒	测试写成 `func TestX(t *testing.T){}` → 骨架上全绿 → 期望 `tests_not_red`
4	越界写被拒	提交 `src/main.go`（不在 writes 里）→ 期望 `path_outside_writes`
5	产物可追溯	`openvow task show T-3` 能列出每个文件的 sha、terminal、attempt、时间

第 3 条是 M0 的灵魂。 它证明"判卷的人不是考生"这条原则不是口号，而是能被执行器机械判定的。

---

11. 测试计划

层	内容	数量级
单测	路径规范化、glob、哈希、租约计算	30 个用例
单测	流水线六步的拒绝分支	每码 2 例（正/反）
集成	`httptest` 跑 6 端点	12 例
端到端	验收脚本里的 5 条	5 例
对抗	超大文件、重复提交、过期租约、篡改哈希	8 例

---

12. 十天排期

天	事	出口
D1	仓库骨架、`serve` 起得来、迁移跑通	`sqlite3 openvow.db .tables` 有 6 张表
D2–3	契约层（S2）+ 单测	20 个路径用例全绿

天	事	出口
D4–6	提交流水线（S3）	六步顺序正确，六个拒绝码可复现
D7	HTTP 端点（S4a）	curl 能 claim / submit
D8	CLI（S4b）	命令行走完一轮
D9	验收脚本（S5）+ demo 项目	五条 PASS
D10	修边界、写 README、录一次完整演示	新人照 README 能跑起来

---

13. M0 明确不做

不做	何时做
图（graph）、planner、人审	M1
多终端、并行认领、写集互斥 SQL	M2
租约自动续期、检查点、断点续跑	M2
Postgres、`SKIP LOCKED`	M2
容器隔离（**M0 在当前机器上跑验收命令**）	M3
信任分、抽检	M2
integration / repair 任务	M3
MCP 适配器	M2（有了并行才需要）

安全提醒：M0 在宿主机上执行 `acceptance` 命令，等于执行终端提交的代码。**只在你信任的仓库和终端上跑**，不要接入外部不可信来源。容器化是 M3 的事，在那之前别把它部署到共享环境。

---

14. 完成之后先做什么

1. 用一个**真实的小需求**（不是 demo）跑一遍：3–5 个任务，看返工发生在哪一步。
2. 把返工原因记下来——这些就是 M1 里 V 组校验的规则来源。
3. 如果真实需求跑下来返工率 > 40%，先别急着做 M1 的图，回头把**契约字段**改对（大概率是 `writes` 切得太粗或 `acceptance` 写得太虚）。

---

15. 一句话

M0 只证明一件事：一个 Agent 交上来的东西，能不能被判卷——而判卷的人，不是它自己。 这一条立住了，后面加多少并发都是工程问题；这一条立不住，加多少并发都是灾难。